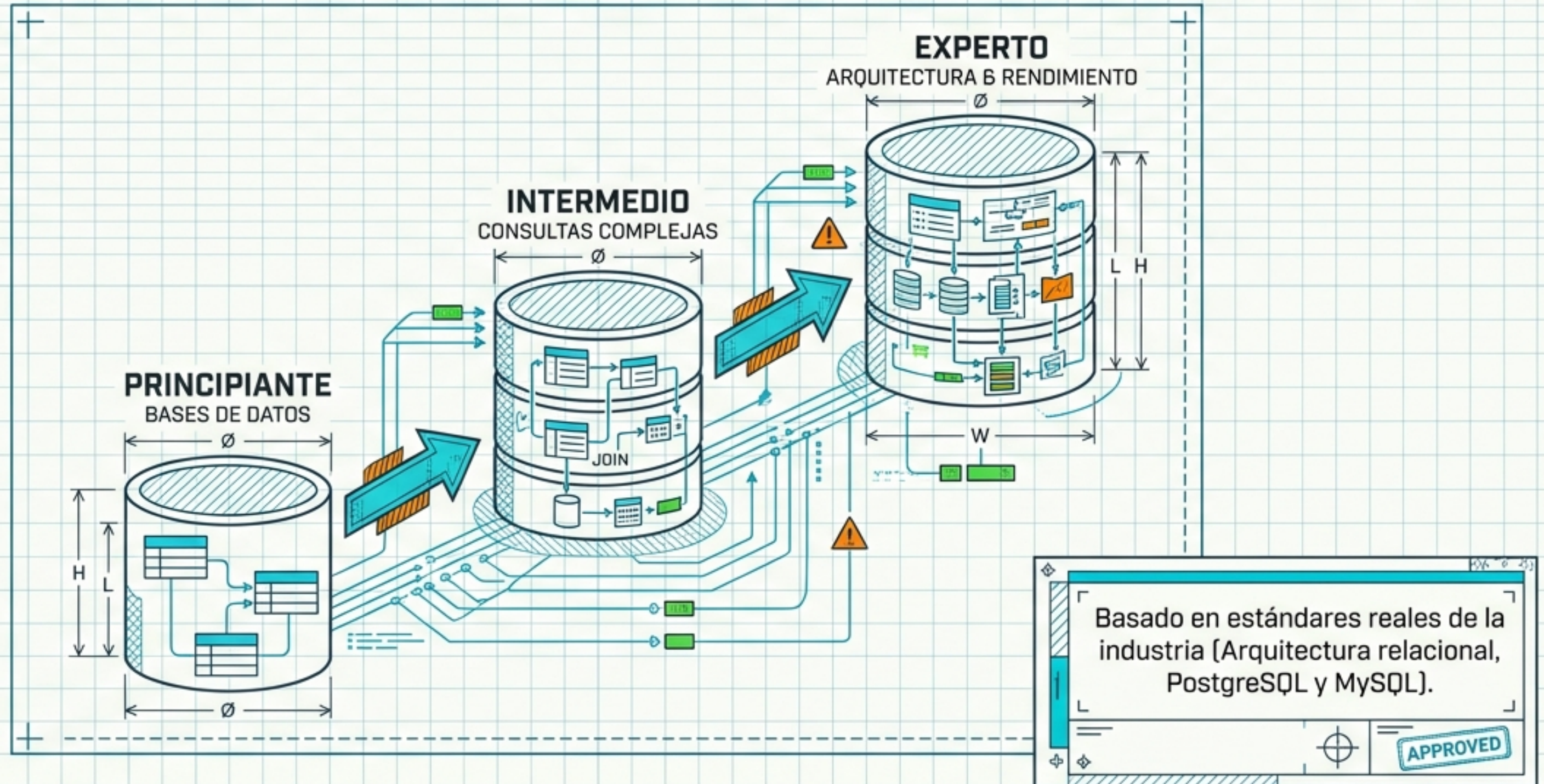


SQL: De Principiante a Experto

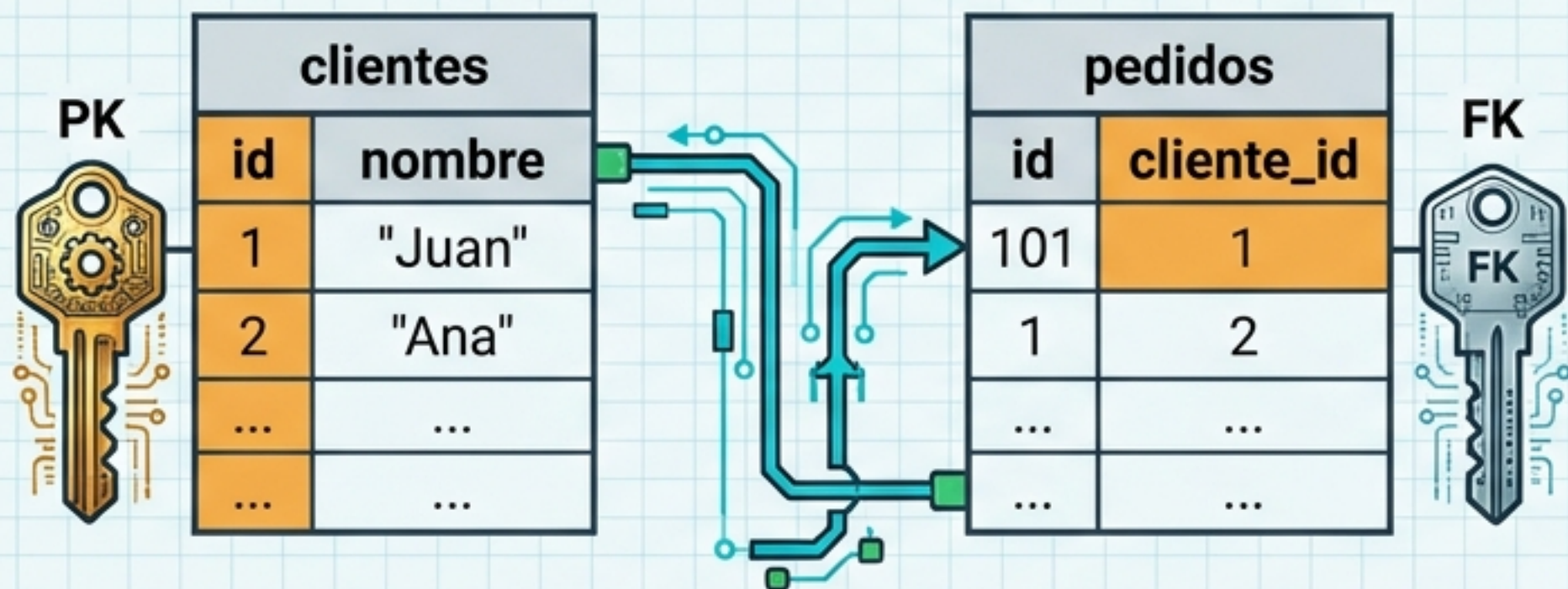
La guía definitiva y **progresiva** para dominar el lenguaje universal de los datos.



BLOQUE 1 - FUNDAMENTOS (Principiante) | Los Cimientos: Tablas y Relaciones

¿Qué es una Base de Datos Relacional?

Información estructurada en **Tablas** (cuadrículas), compuestas por **Columnas** (atributos) y **Filas** (registros).



El ADN de las relaciones:

- **Primary Key (PK):** Identificador único de cada fila. Imposible duplicar.
- **Foreign Key (FK):** El puente. Referencia a la PK de otra tabla.

Caja de Herramientas SQL



DQL (Leer):

```
SELECT *  
FROM clientes;
```



DML (Modificar datos):

```
INSERT INTO clientes  
(nombre)  
VALUES ('Ana');
```



DDL (Estructura):

```
CREATE TABLE pedidos  
(id INT);
```



DCL (Permisos):

```
GRANT SELECT ON  
clientes TO admin;
```

Tipos de datos comunes: INT (Enteros), VARCHAR (Texto), DATE (Fechas), DECIMAL (Dinero).

BLOQUE 1 (Principiante) | Resumen Rápido: Fundamentos

Checklist de Validación

- ✓ **SQL se divide por propósito:** DDL (estructuras), DML (datos), DQL (consultas) y DCL (seguridad).
- ✓ **Tablas:** Son las matrices donde viven nuestros datos.
- ✓ **Clave Primaria (PK):** Asegura que no existan dos filas idénticas. Es la huella digital del registro.
- ✓ **Clave Foránea (FK):** Permite construir relaciones entre distintas tablas sin duplicar información.
- ✓ **Tipos de Datos:** Elegir correctamente (INT vs VARCHAR) optimiza el espacio y evita errores críticos.

Nota del Ingeniero

Comparativa: Al crear IDs automáticos, PostgreSQL usa SERIAL, mientras que MySQL usa AUTO_INCREMENT.

BLOQUE 2 - CONSULTAS BÁSICAS (Principiante-Intermedio) | El Poder del SELECT

Orden de Escritura

SELECT → FROM → WHERE → ORDER BY → LIMIT

Orden de Ejecución
del Motor

→ FROM → WHERE → SELECT → ORDER BY → LIMIT

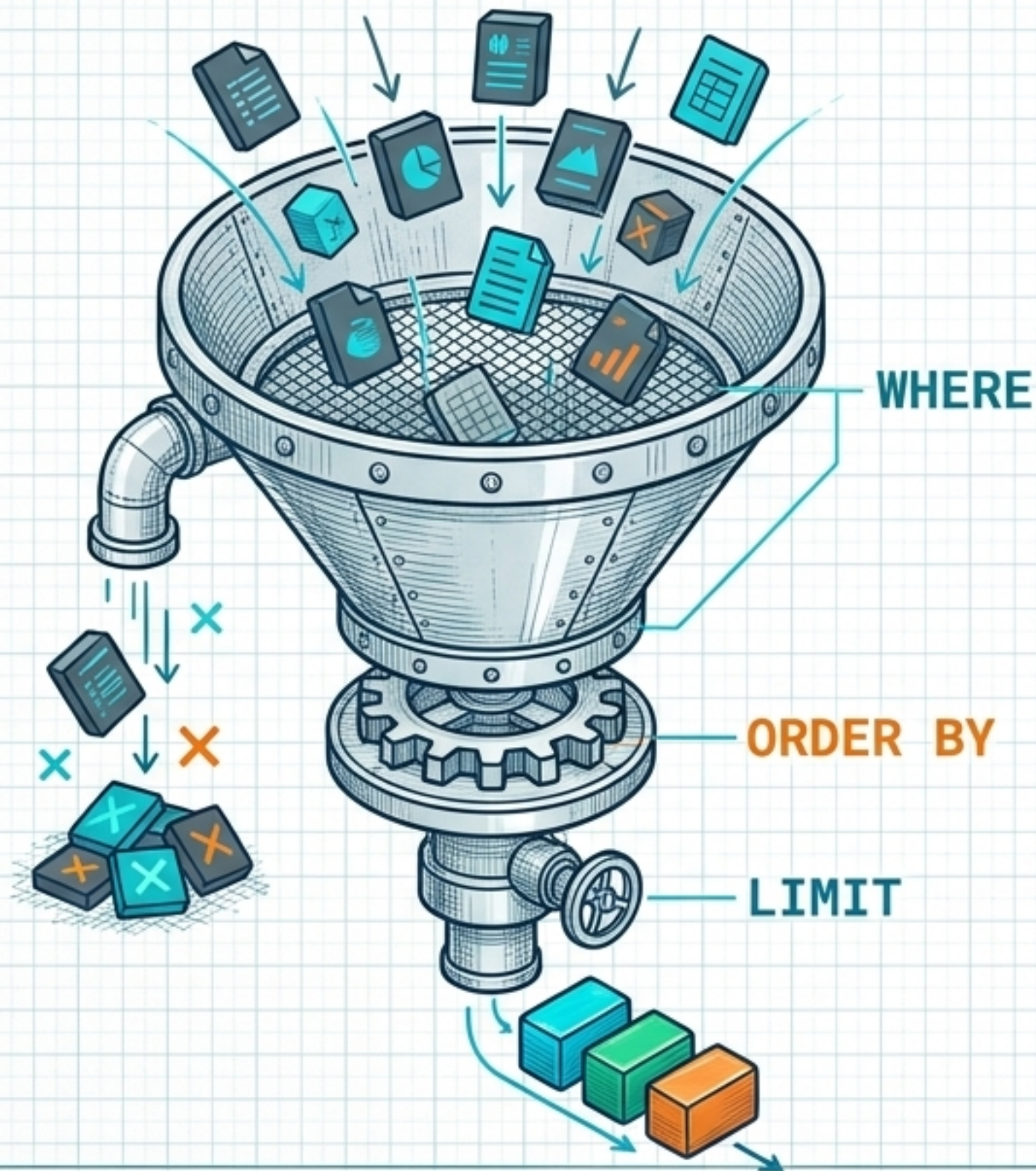
```
SELECT nombre, pais -- 3. Proyecta: Dime qué columnas quiero ver
FROM clientes       -- 1. Origen: Ve a la tabla 'clientes'
WHERE pais = 'ES'   -- 2. Filtra: Solo los de España
    AND nombre LIKE 'A%' -- y cuyo nombre empiece por la letra 'A'
ORDER BY nombre DESC -- 4. Ordena: De la Z a la A
LIMIT 5;            -- 5. Restringe: Devuélveme solo los primeros 5
```

Herramientas de Filtrado

Lógicos: AND (ambas se cumplen), OR (una se cumple), NOT (inversión).

Comodines (LIKE): % (cero o más caracteres), _ (un solo carácter exacto).

BLOQUE 2 (Principiante-Intermedio) | Resumen Rápido: Consultas Básicas

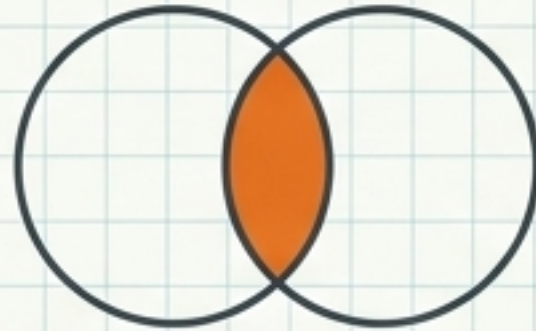


Puntos Clave del Procesamiento:

- La regla de oro del **FROM**: El motor lee el **FROM** antes que el **SELECT**. Por eso no puedes usar un alias definido en el **SELECT** dentro de tu **WHERE**.
- El guardián (**WHERE**): Evalúa fila por fila; descarta todo lo que no cumple la condición lógica antes de procesarlo.
- El costo del **ORDER BY**: Ordenar datos consume memoria y CPU. Úsalo solo si la aplicación final realmente necesita los datos ordenados.
- Exploración segura (**LIMIT**): Vital para inspeccionar tablas de millones de registros sin colapsar la memoria del servidor.

BLOQUE 3 - RELACIONES (Intermedio) | Ensamblando Datos: Los 4 JOINS

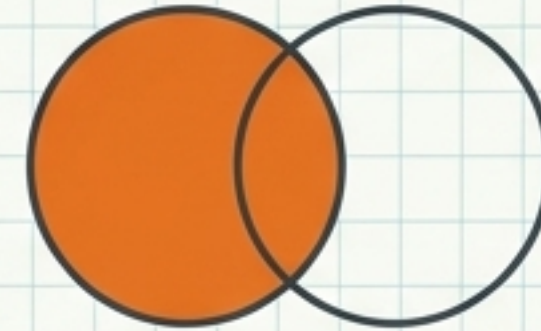
INNER JOIN (Intersección)



```
SELECT c.nombre, p.monto FROM clientes c  
INNER JOIN pedidos p ON c.id = p.cliente_id;
```

Resultado: Solo coincidencias exactas. Solo clientes que tienen pedidos.

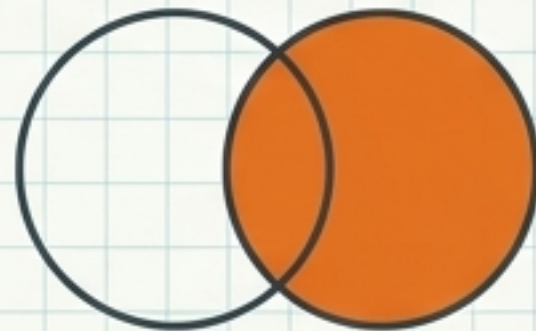
LEFT JOIN (Todo lo izquierdo)



La tabla base manda.

Resultado: Muestra a todos los clientes. Si no tienen pedidos, la columna monto mostrará NULL.

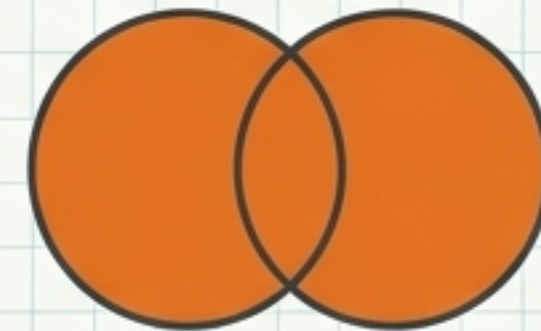
RIGHT JOIN (Todo lo derecho)



La tabla secundaria manda.

Resultado: Muestra todos los pedidos, incluso si el cliente original fue eliminado (NULL en cliente).

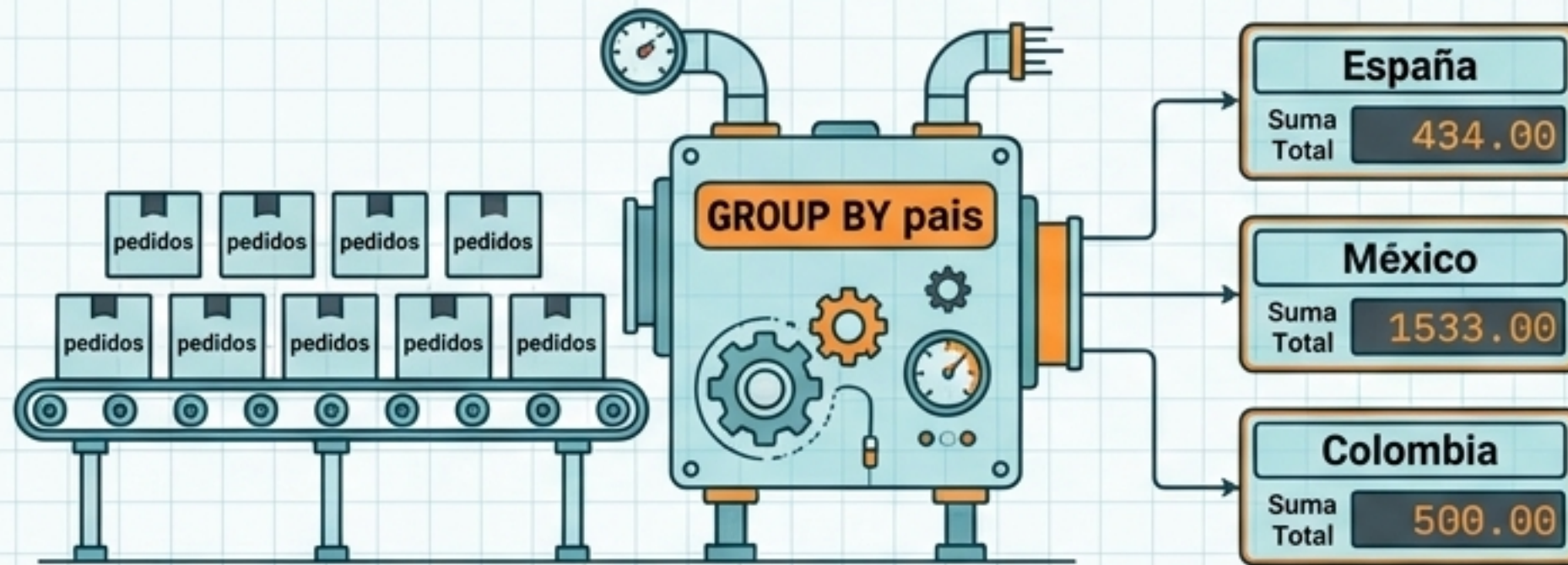
FULL JOIN (El total)



Combina ambos mundos.

Resultado: Trae todo de ambas tablas, cruzando donde coincidan y rellenando con NULL donde no.

BLOQUE 3 (Intermedio) | Agrupación y Resumen Rápido



Funciones de Agregación: Aplastan múltiples filas en un solo valor numérico (SUM, AVG, COUNT, MAX, MIN).

WHERE vs HAVING: La batalla del filtrado.
WHERE filtra datos crudos antes de agrupar.
HAVING filtra los bloques después de agrupar.

CÓDIGO SQL

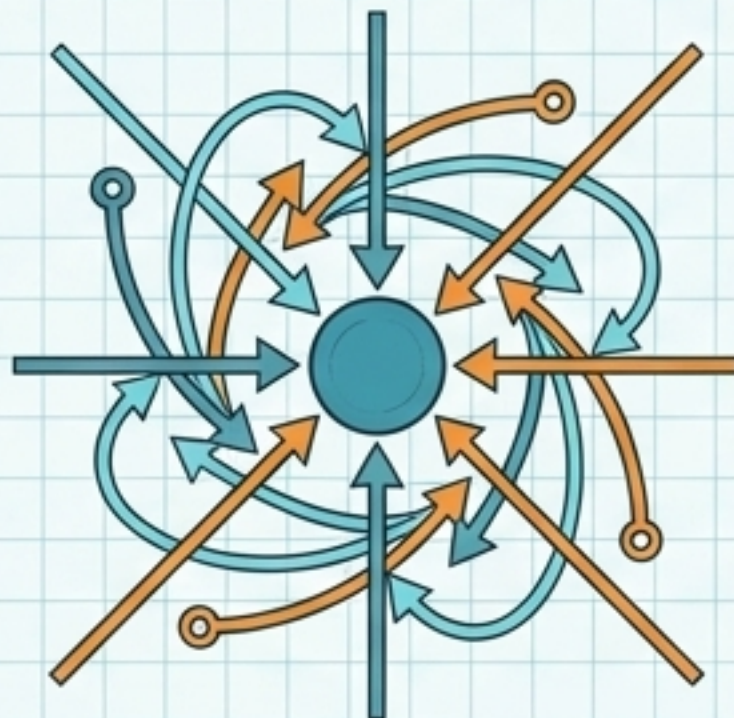
```
SELECT c.pais, SUM(p.monto) as total_ventas
FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
WHERE c.estatus = 'Activo'           -- 1. Filtra clientes activos
GROUP BY c.pais                     -- 2. Agrupa los restantes por país
HAVING SUM(p.monto) > 5000;          -- 3. Descarta grupos con ventas bajas
```

Checklist de Validación

- ✓ **JOIN** combina columnas de diferentes tablas usando puentes (FK=PK).
- ✓ **GROUP BY** condensa filas basándose en valores idénticos.
- ✓ **Regla Estricta:** Toda columna en el SELECT que no tenga una función de agregación, debe estar listada en el GROUP BY.

BLOQUE 4 - CONSULTAS AVANZADAS (Intermedio-Avanzado) | CTEs y Subconsultas

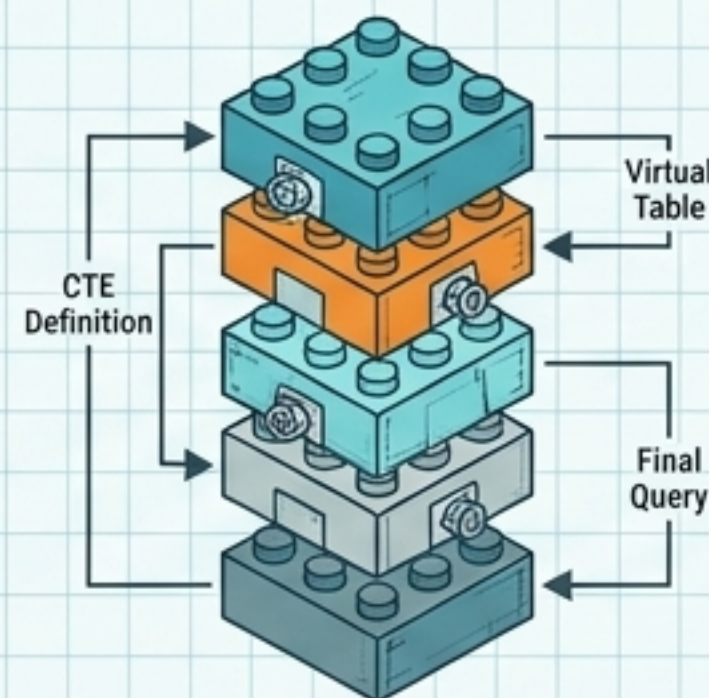
Subconsultas (El Método Clásico)



Consultas dentro de otras. Útiles, pero difíciles de leer si anidas demasiadas.

```
-- Clientes con compras usando EXISTS (más eficiente que IN para tablas grandes)
SELECT nombre FROM clientes c
WHERE EXISTS (
    SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id
);
```

CTE - Common Table Expressions (El Método Moderno)



Usa la cláusula **WITH** para crear variables lógicas o “tablas temporales virtuales” al inicio de tu script. Hacen el código modular, limpio e incluso recursivo.

```
WITH ClientesVIP AS (
    SELECT cliente_id, SUM(monto) as total
    FROM pedidos GROUP BY cliente_id HAVING SUM(monto) > 1000
)
-- Ahora leemos de la CTE como si fuera una tabla real:
SELECT c.nombre FROM clientes c
JOIN ClientesVIP v ON c.id = v.cliente_id;
```

BLOQUE 4 (Intermedio-Avanzado) | Funciones de Ventana y Resumen Rápido

El Superpoder del OVER()

Antes

Tabla Pedidos Detallada				
id	nombre	país	fecha	monto
101	Cliente A	España	2023-01-05	150
102	Cliente B	México	2023-01-12	200
103	Cliente A	España	2023-01-18	120
104	Cliente C	Colombia	2023-01-20	300
105	Cliente B	México	2023-01-25	250
106	Cliente C	Colombia	2023-01-28	180

Medio

Tabla Pedidos Detallada				
id	nombre	país	fecha	monto
101	Cliente A	España	2023-01-05	150
102	Cliente B	México	2023-01-12	200
103	Cliente A	España	2023-01-18	120
104	Cliente C	Colombia	2023-01-20	300
105	Cliente B	México	2023-01-25	250
106	Cliente C	Colombia	2023-01-28	180

Ventana (PARTITION BY país)

Ventana (PARTITION BY país)

Después

Tabla con Funciones de Ventana					
id	nombre	país	fecha	monto	Ranking
101	Cliente A	España	2023-01-05	150	1
102	Cliente B	México	2023-01-12	200	2
103	Cliente A	España	2023-01-18	120	2
104	Cliente C	Colombia	2023-01-20	300	1
105	Cliente B	México	2023-01-25	250	1
106	Cliente C	Colombia	2023-01-28	180	2

A diferencia de GROUP BY (que comprime las filas), las funciones de ventana calculan métricas manteniendo intactas todas las filas originales. Ideal para análisis analítico.

CÓDIGO SQL

```
SELECT nombre, monto,  
       RANK() OVER (PARTITION BY país ORDER BY monto DESC) as ranking_pais,  
       LAG(monto) OVER (ORDER BY fecha) as pedido_anterior  
FROM pedidos;
```

Funciones Clave

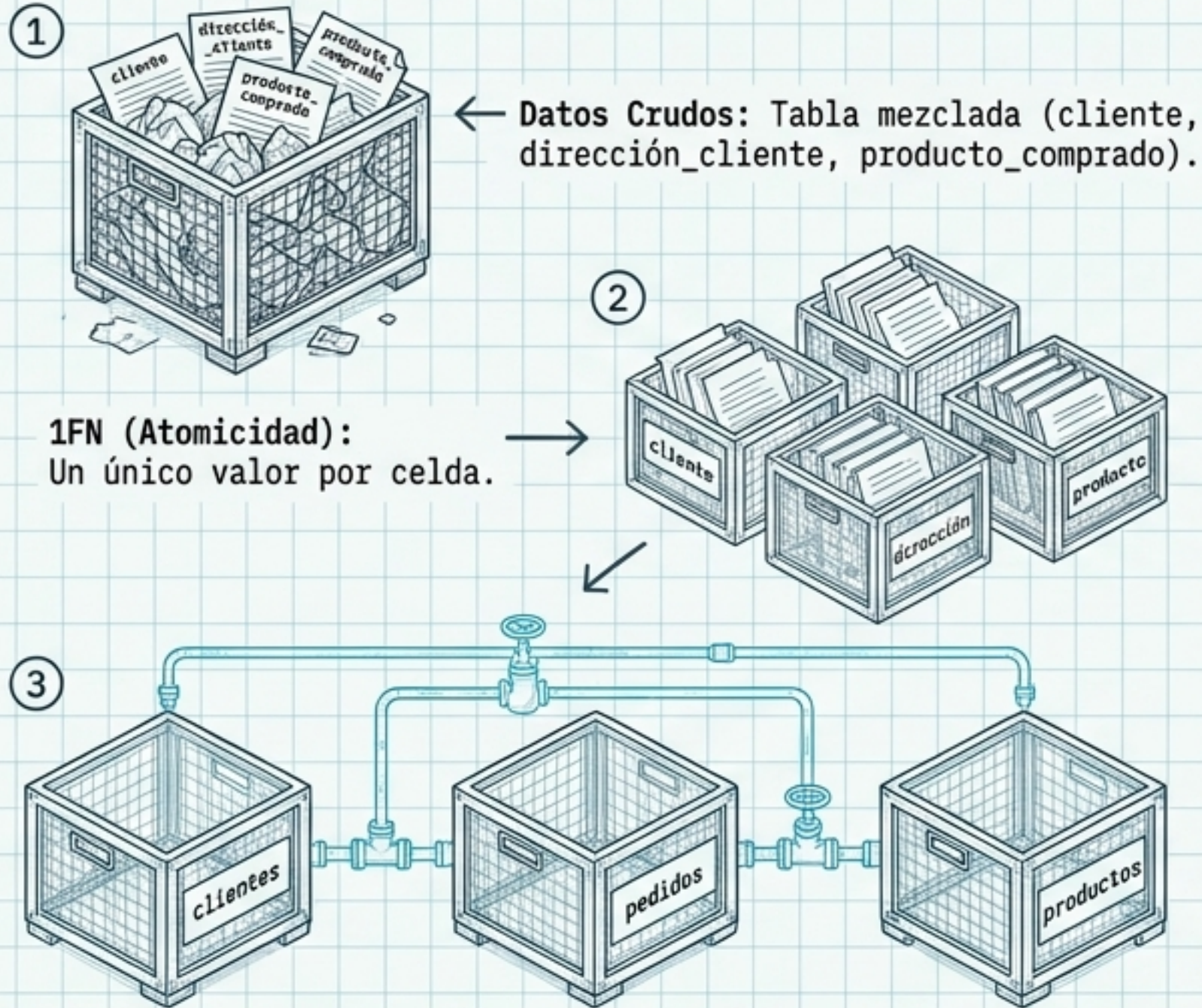
- **ROW_NUMBER / RANK**: Calcula el 'Top N' por categoría.
- **LAG / LEAD**: Compara el registro actual con el registro anterior/siguiente.

Checklist de Validación

- ✓ Usa EXISTS en vez de IN para descartar datos tempranamente.
- ✓ Las CTEs (WITH) son bloques de construcción para código mantenible.
- ✓ Las Window Functions son esenciales para inteligencia de negocios sin destruir el nivel de detalle (granularidad).

BLOQUE 5 - DISEÑO Y OPTIMIZACIÓN (Avanzado) | Normalización e Índices

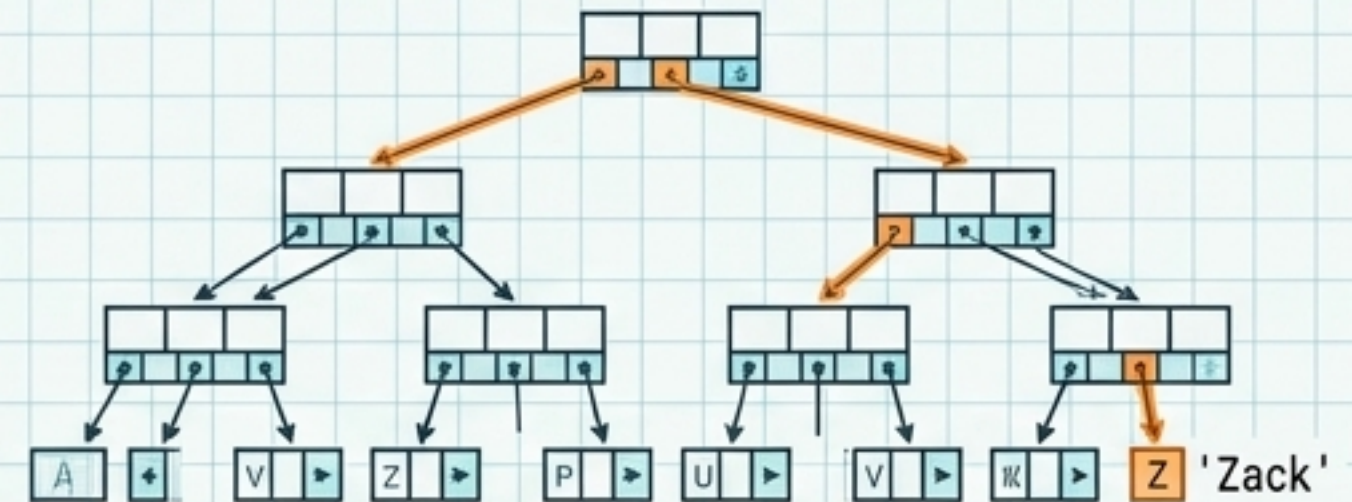
Normalización (Limpiando el Caos)



Índices (La Vía Rápida)



- Full Table Scan: Buscar a 'Zack' leyendo la tabla fila por fila.



- B-Tree Index: El motor navega por un árbol de nodos directo a la letra Z.

Advertencia de Ingeniería: Aceleran los SELECT (lectura), pero ralentizan los INSERT/UPDATE (escritura, porque el árbol debe reconstruirse). ¡No indexes cada columna!



BLOQUE 5 (Avanzado) | Análisis de Ejecución (EXPLAIN) y Resumen



Rayos X a tu Consulta: Nunca adivines por qué un query es lento. Antepón la palabra EXPLAIN (o EXPLAIN ANALYZE en PostgreSQL) a tu consulta para leer el plan de vuelo del motor.

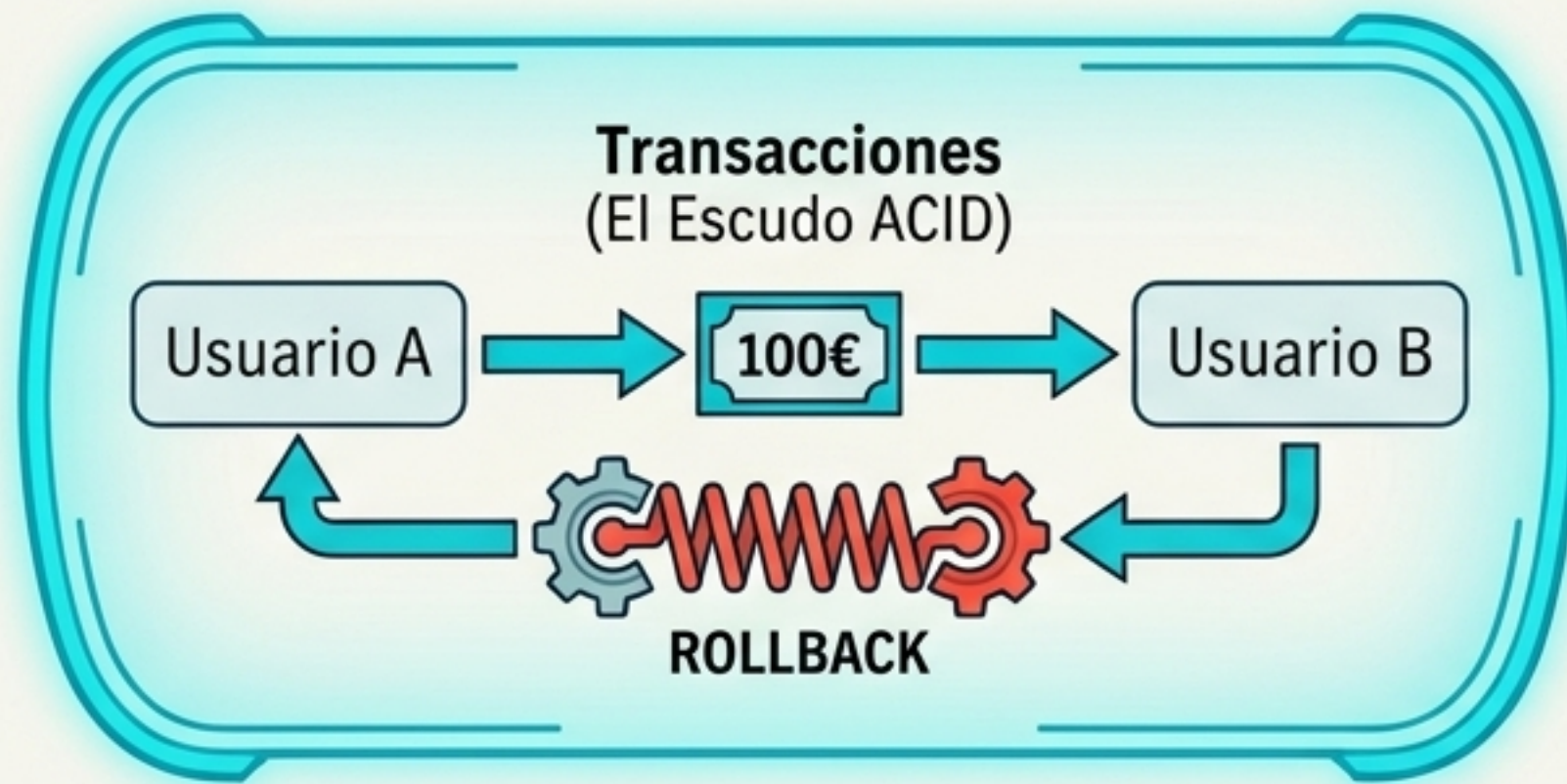
CÓDIGO SQL

```
EXPLAIN
SELECT *
FROM clientes
WHERE pais = 'ES';
```

- ⚠ **Seq Scan (Sequential Scan):** El motor leyó toda la tabla. Malo en tablas gigantes. Falta un índice o la estadística asume que devolverá casi todos los datos.
- ✅ **Index Scan / Seek:** El motor usó un índice para ir directo al dato.

Checklist de Validación

- ✓ Normalizar es crear tablas temáticas específicas para evitar datos duplicados.
- ✓ Indexar estratégicamente las columnas usadas en WHERE o JOIN.
- ✓ Auditar siempre consultas críticas con EXPLAIN.



Transacciones (El Escudo ACID): Atomicidad (Todo o Nada), Consistencia, Aislamiento, Durabilidad. Protegen los datos contra fallos a mitad de proceso.

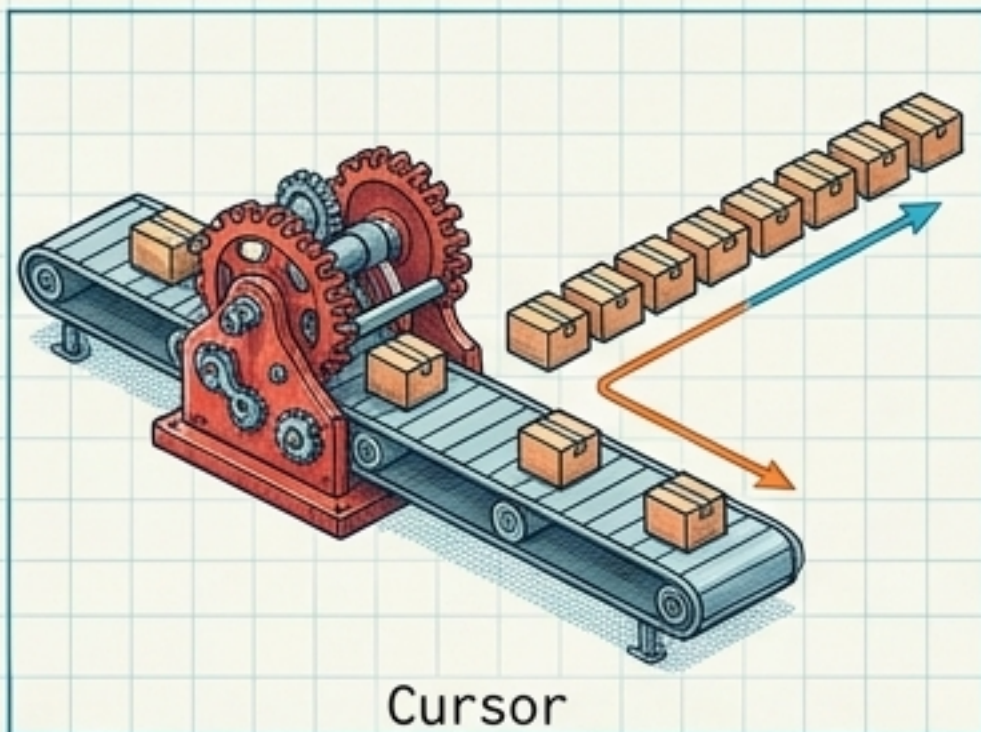
```
BEGIN; -- Inicia bloque seguro (START TRANSACTION en MySQL)
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 'A';
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 'B';
COMMIT; -- Guarda todo (o ROLLBACK si hay error)
```

Lógica en la Base de Datos:

- **Procedimientos Almacenados (SPs):** Scripts pre-compilados en el servidor para ejecutar procesos de negocio complejos (procesamiento por lotes).
- **Funciones:** Rutinas que devuelven un valor calculado (pueden usarse dentro de un SELECT).
- **Triggers (Disparadores):** Código que salta automáticamente al ocurrir un INSERT, UPDATE o DELETE (ej. crear tablas de auditoría).



BLOQUE 6 (Experto) | Seguridad, Cursores y Resumen Rápido



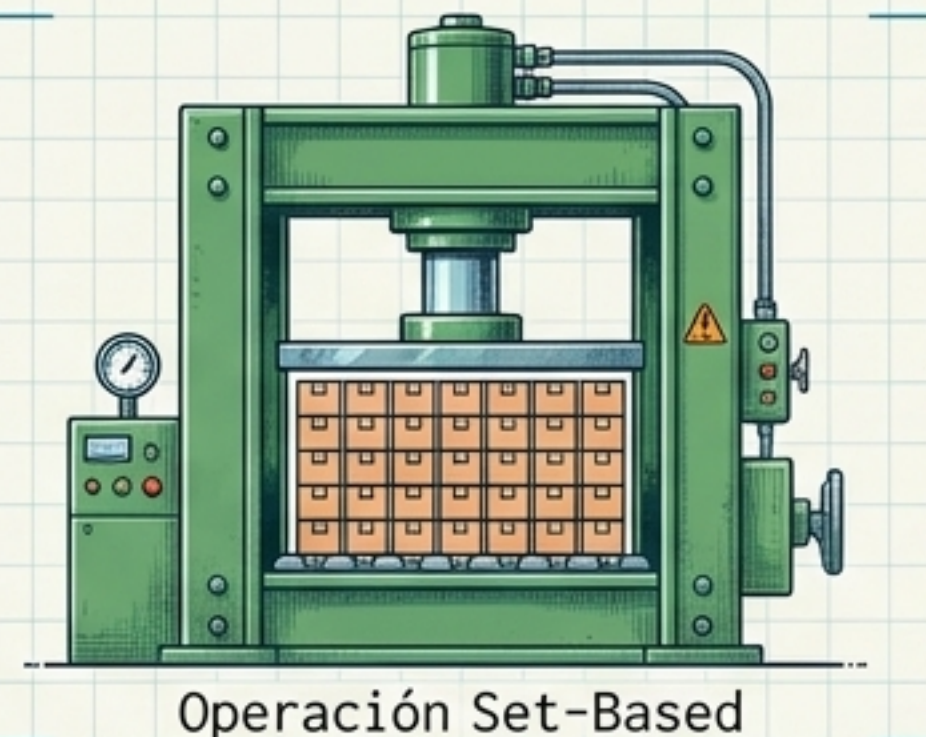
El Antipatrón de los Cursores:

Los cursores (DECLARE, OPEN, FETCH, CLOSE) obligan al motor a trabajar fila por fila. En SQL, esto es destructivo para el rendimiento.

¡Piensa en conjuntos (Set-based), no en bucles (loops)!
Usa actualizaciones masivas con JOIN.

Checklist de Validación

- ✓ Usa **Transacciones** (BEGIN/COMMIT) al modificar múltiples tablas.
- ✓ Los **Triggers** automatizan respuestas internas.
- ✓ El verdadero experto optimiza evitando cursores y manipulando conjuntos enteros de datos.



Seguridad Activa:

- **Mínimo Privilegio:** Limita roles.
GRANT SELECT ON clientes TO analista;
- **Inyección SQL:** Nunca concatenes strings de usuarios directamente. Usa siempre consultas parametrizadas en el backend de tu aplicación.

BLOQUE 7 - CIERRE | Tu Roadmap SQL (El Viaje del Héroe de los Datos)

7

Maestría: Blindaje de operaciones. Orquestas lógicas masivas sin cursores y garantizas consistencia absoluta mediante transacciones **ACID**.

Arquitectura: Diseño estructural. Normalizas en 3FN, **3FN**, **índices B-Tree** y auditas rendimiento con **EXPLAIN**.

Consultas Básicas: Mecánica de filtrado. Comprendes el orden de ejecución real (**FROM** -> **SELECT**) y aislas datos con **WHERE**.

Avanzado: Refinamiento analítico. Modularizas código con **CTEs** y dominas el tiempo y rankings con **Window Functions**.

Relaciones: Ensamblaje. Conectas universos aislados usando los 4 tipos de **JOIN** y extraes métricas clave con **GROUP BY**.

Fundamentos: Construcción de cimientos. Entiendes **tablas**, **PKs/FKs** y dominas el vocabulario base (**DDL**, **DML**).



BLOQUE 7 - CIERRE | SQL Cheat Sheet Definitiva

Estructura Base (DDL / DML)

```
CREATE TABLE x (id INT PRIMARY KEY,  
nom VARCHAR(50));
```

```
INSERT INTO x (id, nom) VALUES  
(1, 'Alpha');
```

Las 3 Reglas de Oro del Taller

1. **Filtra lógicas de relación** en el **ON**; **filtra reglas de negocio** en el **WHERE**.
2. Usa **EXISTS** sobre **IN** para subconsultas de gran volumen.
3. Nunca uses **SELECT *** en entornos de producción. Declara explícitamente tus columnas.

Anatomía de la Consulta Maestra

```
WITH datos AS (SELECT * FROM tabla_base) - 1. Tablas temporales/CTEs  
  
SELECT col1, SUM(valor) OVER() ← 6. Proyección y Funciones de Ventana  
  
FROM datos d ← 2. Origen principal  
  
INNER JOIN otra_tabla o ← 3. Cruces y relaciones  
ON d.id = o.fk_id  
  
WHERE d.fecha >= '2024-01-01' ← 4. Filtrado pre-agrupación  
  
GROUP BY col1 ← 5. Agrupación analítica  
  
HAVING SUM(valor) > 100 ← 7. Filtrado post-agrupación  
  
ORDER BY 2 DESC ← 8. Ordenamiento final  
  
LIMIT 10; ← 9. Restricción de salida
```